# Admin SPA on messaging's staff routes

Report of the work done on branch feat/gateway-api, 11 October 2026

| | |
|---|---|
| Status | Done and verified; committed with this report. |
| Scope | The support queue and the bulletins screens of the admin SPA (`gateway-admin`) now read messaging's staff routes and WebSockets through the admin edge instead of local mock data, and a staff conversation screen was added so a staff member can reply after joining. No edge or backend change. |
| Tests | 65 SPA tests pass (47 before), type check, lint and translation parity (493 keys in en, da, sv) green; the admin jar with the SPA built and run against messaging; checklist in section 5. |

## 1. What it is about

A member opens a support conversation from the customer app. It waits in status WAITING until a staff member joins it, which turns it OPEN and adds the staff member as a participant; from then on staff read and post through the same conversation routes a member uses, plus a live WebSocket per conversation. A bulletin is a short text a staff member posts for an audience (staff or members), optionally limited to a client type and with an expiry; it is stored as a row and pushed live to every staff dashboard connected on the staff channel `/ws/staff`.

The two admin screens for this were built on a sketch of messaging written before the module existed and only ever ran on the service worker's fixtures. Messaging ships the real thing (`AdminMessagingController`, `ConversationController`, `/ws/staff`, `/ws/conversations/{id}`) and the admin edge routes it since the staff identity round. Beyond the field names, one gap mattered: after joining, a staff member had nowhere to talk. The admin SPA had no conversation screen.

## 2. Decisions

| Decision | Consequence |
|---|---|
| Types follow messaging's views | The client declares `ConversationView`, `MessageView`, `ParticipantView`, `BulletinView` and the page envelope field for field. The queue shows subject, member, opened time, last activity with the message count and the staff who joined; there is no preview text in the backend, so there is none on the screen. |
| One status per call, no "All" | WAITING by default, then OPEN, CLOSED, BLOCKED; the page polls every 15 seconds because nothing pushes queue changes. Messaging's page carries no total, so `Pagination` gained an unknown-total mode (Next while a full page came back). |
| A staff conversation screen at `/support/:id` | Join opens it. Row and history over REST, then the conversation socket through the edge; frames `message`, `status`, `ack`, `error` handled; text composer over the socket with a REST fallback and polling while the socket is down; read marker; join, close and block with a reason. The socket hook is a port of the customer SPA's. |
| Bulletins per the backend | Form: content (first line is the heading, at most 1023 characters), audience STAFF / MEMBERS, client type every / web / mobile sent lowercase with empty for every, optional expiry. List per audience, newest first; the page listens on `/ws/staff?clientType=web` and prepends a bulletin that arrives live. The title field went: the backend has none. |
| Flags as in the trade round | `features.chat` defaults on; the worker starts only with `features.mocks`; the chat fixtures were rewritten to the real shapes and rules (status filter, size cap, no total, join idempotence, 409 on closed or blocked, 403 before joining). |

## 3. What changed

### Admin SPA (`gateway-admin/src/main/webapp`)

| Area | Files | Change |
|---|---|---|
| Client | `app/shared/api/supportApi.ts` | Row and frame types, `messageOfFrame` / `bulletinOfFrame` (cents to amounts, frame to view), `socketUrl`, the queue, conversation, message, read, join, close, block and bulletin endpoints; mutations refetch through the tags. |
| Socket | `app/modules/chat/useSocket.ts` | One hook for both channels: token from the store as `?access_token=`, reconnect with backoff, `unavailable` after three failures, `send` answers false when the socket is not open. |
| Screens | `SupportQueuePage.tsx`, `SupportThreadPage.tsx` (new), `BulletinsPage.tsx` | Queue per decision 2 with a link per row and a Join button on WAITING rows; the conversation screen per decision 3; bulletins per decision 4. |
| Shared | `Pagination.tsx`, `routes.tsx`, `features.ts`, `index.tsx` | Unknown-total mode; the `/support/:id` route; chat flag on; bootstrap comment. |
| Mocks | `app/mocks/chatFixtures.ts` (new), `handlers.ts` | Fixtures and handlers in messaging's shapes and rules; join, close and block change the row only. |
| Translations | `app/i18n/{en,da,sv}/{chat,common}.json` | Queue, conversation, socket states, bulletins, audiences, client types, the page summary without total. |
| Tests | `app/test/{testUtils,fakeWebSocket}.tsx\|ts`, three page tests, `chatHelpers.test.ts` | The trade test utilities moved to `app/test` and are shared; a WebSocket double records the URL and sent frames and lets a test push server frames. |

### Documentation

- `doc/frontend-edges.md` §6, §11 and §13 touched, new §14 (what was wired, the screen, flags, tests) and §14a (checklist).
- Memory of the plan written as done.

## 4. Verification

| Check | Result |
|---|---|
| `npm run typecheck`, `npm run lint` | clean |
| `npm run i18n:check` | OK: 11 files, 493 keys in en, 460 literal keys in code, da and sv in parity |
| `npm test` | 17 suites, 65 tests pass (47 before this work) |
| `mvn -pl db-reactive,cqrs-reactive,gateway-admin -P webapp -DskipTests package` | jar serves the SPA with the chat flag on |

## 5. Checklist against the running services

Setup: live MySQL, `docker/cqrs` Kafka 9094, `gateway-admin` from the jar above with `--spring.profiles.active=dev` on 8081, messaging from `mvn -pl messaging spring-boot:run` on 8086 after installing the sibling modules from this worktree. Two deviations: the development EventSourcingDB on port 3000 still hangs on every streaming read (ping 200, read-events never answers, the state noted on 10 October), so messaging ran against a throwaway `eventsourcingdb:1.2.0` on port 3001 with a temporary data directory, its four read-side checkpoint rows deleted before and restored after the run; and messaging ran with the service token off so the member side could be driven with curl directly. Every staff call went through the admin edge; screens driven in headless Chromium. The three support conversations and two bulletins the run wrote were deleted from the development database afterwards.

| Step | Result |
|---|---|
| Member opens a support conversation and posts | 201 both; `GET /api/admin/conversations?status=WAITING` through the edge answers the row with subject, one USER participant, a page without total |
| Support queue screen | one WAITING row: subject, member #88 linking to the member, opened time, "1 messages", Waiting badge, Join; "Page 1 · 1 rows on this page"; no alert, no mock banner |
| Join from the queue | navigates to `/support/{id}`; header "Open · Member #88 · Staff #1 · Opened …", composer shown; the WAITING list is empty afterwards, the OPEN list holds the row |
| Conversation screen | history rendered, socket badge "Live" on `ws://localhost:8081/ws/conversations/{id}?access_token=…`, `read` frame sent |
| Staff reply through the socket | "Hello, I am looking at the payment now." appears as "You" (pushed back through `messaging.live`) |
| Member reply over REST while the screen is open | arrives live in 66 ms; the member's message list holds all three messages |
| Bulletins screen | socket "Live" on `/ws/staff?clientType=web`; posting content with client type web: "Bulletin posted.", one row with heading, body, "Web", author #1 |
| Bulletin posted by another client through the edge | prepended live in 94 ms; list newest first |
| Close from the screen | footer "This conversation is Closed.", badge Closed, Close and Block gone; `/support?status=CLOSED` lists it |
| Block with a reason (edge) | 200; the row reads BLOCKED with the reason |
| Rules | join on the closed conversation 409 `conversation-closed` (problem+json); a member post into it 409 as well |
| Console | one expected 401 (the silent refresh before login); nothing else |

## 6. Observations and what is left

- **Staff actions post no message.** Join, close and block push a `status` frame; SYSTEM messages come from order events only. The first fixtures assumed a system line per action and were corrected after the run.
- **The development EventSourcingDB needs a restart.** Port 3000 answers the ping but hangs on reads since 10 October; this session was not allowed to restart the container, so the run used a throwaway store. `docker restart announce_esdb` should bring it back.
- **Live frames are not mocked in the browser.** With `features.mocks` the screens poll; the page tests cover the frames through the double.
- Roles are still not checked inside the backends; the edge enforces `ROLE_ADMIN` on `/api/**` and `/ws/**`.
- The customer SPA's chat still has the WebSocket mock on its list (frontend-edges §9); that side is unchanged by this round.